

Описание функций сглаживания — Программистская версия

Документ описывает четыре функции сглаживания: их сигнатуры, входные/выходные данные, алгоритмы и особенности реализации.

1. Boxcar (Равномерное скользящее среднее)

1.1 Сигнатура

Python — Boxcar.py

```
def boxcar(win: int, data: list[float]) -> list[float]:
```

C++ — calculator.h

```
inline void movingAverage(const vector<double>& in,
                          vector<double>& out, int win);
```

1.2 Параметры

Параметр	Тип	Описание
win	int	Размер окна усреднения
data / in	list / vector<double>	Входной сигнал длины N
out	list / vector<double>	Выходной сигнал длины N

1.3 Алгоритм (Python)

Boxcar.py:2–12

```
def boxcar(win, data):
    half = win // 2
    n = len(data)
    avg = []
    for i in range(n):
        start = max(0, i + 1 - half)
        end = min(half + i, n)
        sum_array = sum(data[start:end])
        aver = sum_array / len(data[start:end])
        avg.append(aver)
    return avg
```

Логика: для каждого индекса i вычисляются границы окна $[start, end)$ с обрезкой по краям массива. Вычисляется арифметическое среднее по срезу $data[start:end]$.

1.4 Алгоритм (C++)

calculator.h:31–50

```
// 1. Дополнение отражением (mirror padding)
vector<double> padded(n + 2 * half);
for (int i = 0; i < half; i++) {
    padded[i] = in[half - 1 - i];
    padded[n + half + i] = in[n - 1 - i];
}
for (int i = 0; i < n; i++)
    padded[half + i] = in[i];

// 2. Префиксные суммы
vector<double> prefix(padded.size() + 1, 0.0);
for (int i = 0; i < (int)padded.size(); i++)
    prefix[i + 1] = prefix[i] + padded[i];

// 3. Вычисление средних
for (int i = 0; i < n; i++)
    out[i] = (prefix[i + win] - prefix[i]) / win;
```

Логика: данные дополняются отражением на half элементов с каждой стороны. Затем строится массив префиксных сумм, что позволяет вычислить сумму любого окна за $O(1)$. Итоговая сложность: $O(n)$ по времени, $O(n)$ по памяти.

1.5 Различия реализаций

Аспект	Python	C++
Граничные условия	Обрезка окна (clamping)	Отражение данных (mirror)
Суммирование	Прямое sum() на срезе	Префиксные суммы
Размер окна на границе	Уменьшается	Всегда = win

2. Gaussian (Гауссов фильтр)

2.1 Сигнатура

Python — Gaussian.py

```
def gaussian(data: list[float], win: int) -> list[float]:
```

C++ — calculator.h

```
vector<double> gaussianKernel(int win);

inline void gaussianFilter(const vector<double>& in,
                          vector<double>& out,
                          const vector<double>& kernel);
```

2.2 Параметры

Параметр	Тип	Описание
win	int	Размер гауссова ядра
data / in	list / vector<double>	Входной сигнал длины N
kernel	list / vector<double>	Нормализованное ядро длины win

2.3 Алгоритм — построение ядра (Python)

Gaussian.py:4–11

```
def gaussian(data, win):
    half = win // 2
    sigma = win / 6.0
    n = len(data)

    # Ядро Гаусса
    k = [math.exp(-0.5 * (i - half)**2 / sigma**2)
          for i in range(win)]
    k = [x / sum(k) for x in k]

    # Дополнение отражением
    p = ([data[i] for i in range(half-1, -1, -1)]
          + data
          + [data[i] for i in range(n-1, n-half-1, -1)])

    # Свёртка
    return [sum(k[j] * p[i+j] for j in range(win))
            for i in range(n)]
```

2.4 Алгоритм — построение ядра (C++)

calculator.h:54–67

```
vector<double> gaussianKernel(int win) {
    int half = win / 2;
    double sigma = win / 6.0;
    vector<double> kernel(win);
    double sum = 0;
    for (int i = 0; i < win; i++) {
        int d = i - half;
        kernel[i] = exp(-0.5 * (d * d) / (sigma * sigma));
        sum += kernel[i];
    }
    for (int i = 0; i < win; i++)
        kernel[i] /= sum;
    return kernel;
}
```

2.5 Свёртка (C++)

calculator.h:70–93

```

inline void gaussianFilter(const vector<double>& in,
                          vector<double>& out,
                          const vector<double>& kernel) {
    int n = (int)in.size();
    int win = (int)kernel.size();
    int half = win / 2;
    out.resize(n);

    // Отражение границ
    vector<double> padded(n + 2 * half);
    for (int i = 0; i < half; i++) {
        padded[i] = in[half - 1 - i];
        padded[n + half + i] = in[n - 1 - i];
    }
    for (int i = 0; i < n; i++)
        padded[half + i] = in[i];

    // Взвешенная свёртка
    for (int i = 0; i < n; i++) {
        double sum = 0;
        for (int j = 0; j < win; j++)
            sum += kernel[j] * padded[i + j];
        out[i] = sum;
    }
}

```

2.6 Сравнение реализаций

Аспект	Python	C++
Ядро и свёртка	В одном вызове	Отдельные функции gaussianKernel + gaussia...
Дополнение	Списковое выражение	Цикл копирования
sigma		
Граничные условия		

3. Progressive (Прогрессивное среднее)

3.1 Сигнатура

Python — Progressive.py

```
def progressive(data: list[float]) -> list[float]:
```

C++ — calculator.h

```

inline void progressiveMean(const vector<double>& in,
                          vector<double>& out);

```

3.2 Алгоритм (Python)

Progressive.py:3–9

```
def progressive(data):
    result = []
    total = 0.0
    for i, value in enumerate(data):
        total += value
        result.append(total / (i + 1))
    return result
```

3.3 Алгоритм (C++)

calculator.h:97–105

```
inline void progressiveMean(const vector<double>& in,
                           vector<double>& out) {
    int n = (int)in.size();
    out.resize(n);
    double sum = 0;
    for (int i = 0; i < n; i++) {
        sum += in[i];
        out[i] = sum / (i + 1);
    }
}
```

3.4 Сравнение реализаций

Аспект	Python	C++
Накопление суммы	total += value	sum += in[i]
Деление	total / (i + 1)	sum / (i + 1)
Реализация		

4. Hybrid (Гибридное сглаживание)

4.1 Сигнатура

Python — Hybrid (Boxcar + Progressive).py

```
def hybrid(data: list[float], win: int) -> list[float]:
```

C++ — calculator.h

```
inline void hybridSmooth(const vector<double>& in,
                        vector<double>& out, int win);
```

4.2 Алгоритм (Python)

Hybrid (Boxcar + Progressive).py:21–37

```
def hybrid(data, win):
    n = len(data)
    box = boxcar(win, data)
    prog = progressive(data)

    out = []
    for i in range(n):
        if i < win:
            out.append(box[i])
        elif i < 2 * win:
            z = (i - win) / win
            k = (1.0 - z) * box[i] + z * prog[i]
            out.append(k)
        else:
            out.append(prog[i])
    return out
```

4.3 Алгоритм (C++)

calculator.h:110–135

```
inline void hybridSmooth(const vector<double>& in,
                        vector<double>& out, int win) {
    int n = (int)in.size();
    out.resize(n);

    vector<double> boxcar;
    movingAverage(in, boxcar, win);

    vector<double> progressive;
    progressiveMean(in, progressive);

    for (int i = 0; i < n; i++) {
        if (i < win) {
            out[i] = boxcar[i];
        } else if (i < 2 * win) {
            double w = (double)(i - win) / win;
            out[i] = (1.0 - w) * boxcar[i] + w * progressive[i];
        } else {
            out[i] = progressive[i];
        }
    }
}
```

4.4 Сравнение реализаций

Аспект	Python	C++
Вызов boxcar	boxcar(win, data)	movingAverage(in, boxcar, win)
Вызов progressive	progressive(data)	progressiveMean(in, progressive)
Коэффициент	$z = (i - \text{win}) / \text{win}$	$w = (\text{double})(i - \text{win}) / \text{win}$
Интерполяция		
Пороги		